

Ski Retail Demand Analysis

Project: Ski Retail Demand vs. Snowfall Forecasting

Author: Chris Maher

Objective: Extract daily NOAA weather metrics for Seven Springs/Somerset County, and pair them with macro market search demand via Google Trends (2021 - Present).

Intro

Unlike western resorts with predictable weather and snowfall, Mid-Atlantic winters fluctuates wildly from season to season, ski retailers cannot rely solely on the calendar to time marketing efforts. By modeling the relationship between snowfall events and local search intent, this analysis turns weather volatility into an operational advantage for ad spend planning and allocation.

```
In [1]: import requests
import pandas as pd
import numpy as np
from pytrends.request import TrendReq
from datetime import datetime
import time
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [2]: #pipeline config
START_DATE = "2021-01-01"
END_DATE = datetime.now().strftime("%Y-%m-%d")

#target geography & search params
GEO_REGION = "US-PA"
KEYWORDS = ["ski shop", "ski tuning", "ski boots", "snowboard boots", "ski rentals"]

#NOAA api config
NOAA_TOKEN = "DFwUbsXdoNuNlLjXTkBmpobepUIALkni"
#Somerset County Airport Station ID (Seven Springs area)
NOAA_STATION_ID = "GHCND:US1PASM0001"

print(f"Pipeline initialized for {GEO_REGION} from {START_DATE} to {END_DATE}")
```

Pipeline initialized for US-PA from 2021-01-01 to 2026-08-26

```
In [3]: def fetch_noaa_data(station_id, token, start_date, end_date):
        """Fetches daily weather metrics from NOAA GHCN API in rate-limited chunks."""
        url = "https://www.ncdc.noaa.gov/cdo-web/api/v2/data"
        headers = {"token": token}
```

```

if not station_id.startswith("GHCND:"):
    station_id = f"GHCND:{station_id}"

start_dt = pd.to_datetime(start_date)
end_dt = pd.to_datetime(end_date)

all_results = []
current_start = start_dt

while current_start < end_dt:
    current_end = min(current_start + pd.Timedelta(days=180), end_dt)

    params = [
        ("datasetid", "GHCND"),
        ("stationid", station_id),
        ("startdate", current_start.strftime("%Y-%m-%d")),
        ("enddate", current_end.strftime("%Y-%m-%d")),
        ("datatypeid", "SNOW"), # Daily snowfall in mm
        ("datatypeid", "PRCP"), # Total precipitation in tenths of mm
        ("limit", 1000)
    ]

    response = requests.get(url, headers=headers, params=params)
    if response.status_code == 200:
        data = response.json().get("results", [])
        all_results.extend(data)
        print(f"Window {current_start.strftime('%Y-%m-%d')} to {current_end.strftime('%Y-%m-%d')}")
    else:
        raise Exception(f"NOAA API Error: {response.status_code} - {response.text}")

    current_start = current_end + pd.Timedelta(days=1)
    time.sleep(0.25)

return pd.DataFrame(all_results)

def fetch_google_trends(keywords, geo, start_date, end_date, retries=3, backoff=5):
    #fetches weekly interest over time scores via pytrends
    pytrends = TrendReq(hl='en-US', tz=300)
    timeframe = f"{start_date} {end_date}"

    for attempt in range(retries):
        try:
            pytrends.build_payload(kw_list=keywords, cat=0, timeframe=timeframe, geo=geo)
            df = pytrends.interest_over_time()
            if 'isPartial' in df.columns:
                df = df.drop(columns=['isPartial'])
            return df
        except Exception as e:
            if "429" in str(e) and attempt < retries - 1:
                wait_time = backoff * (attempt + 1)
                print(f"Google Trends rate limited (429). Retrying in {wait_time}s.")
                time.sleep(wait_time)
            else:
                raise e

```

```
In [4]: #Execute API calls
print("Fetching NOAA weather data...")
df_noaa_raw = fetch_noaa_data(NOAA_STATION_ID, NOAA_TOKEN, START_DATE, END_DATE)

print("Fetching Google Trends data...")
df_trends_raw = fetch_google_trends(KEYWORDS, GEO_REGION, START_DATE, END_DATE)

#Display sanity checks
print("\n--- NOAA Raw Data ---")
print(f"Shape: {df_noaa_raw.shape}")
display(df_noaa_raw.head())

print("\n--- Google Trends Raw Data ---")
print(f"Shape: {df_trends_raw.shape}")
display(df_trends_raw.head())
```

```
Fetching NOAA weather data...
Window 2021-01-01 to 2021-06-30: fetched 277 records
Window 2021-07-01 to 2021-12-28: fetched 274 records
Window 2021-12-29 to 2022-06-27: fetched 286 records
Window 2022-06-28 to 2022-12-25: fetched 279 records
Window 2022-12-26 to 2023-06-24: fetched 293 records
Window 2023-06-25 to 2023-12-22: fetched 286 records
Window 2023-12-23 to 2024-06-20: fetched 288 records
Window 2024-06-21 to 2024-12-18: fetched 304 records
Window 2024-12-19 to 2025-06-17: fetched 321 records
Window 2025-06-18 to 2025-12-15: fetched 362 records
Window 2025-12-16 to 2026-06-14: fetched 359 records
Window 2026-06-15 to 2026-08-26: fetched 140 records
Fetching Google Trends data...
```

```
--- NOAA Raw Data ---
Shape: (3469, 5)
```

	date	datatype	station	attributes	value
0	2021-01-01T00:00:00	PRCP	GHCND:US1PASM0001	„N,0500	183
1	2021-01-01T00:00:00	SNOW	GHCND:US1PASM0001	„N,0500	122
2	2021-01-02T00:00:00	PRCP	GHCND:US1PASM0001	„N,0500	218
3	2021-01-03T00:00:00	PRCP	GHCND:US1PASM0001	„N,0500	5
4	2021-01-04T00:00:00	PRCP	GHCND:US1PASM0001	„N,0500	43

```
--- Google Trends Raw Data ---
Shape: (68, 5)
```

	ski shop	ski tuning	ski boots	snowboard boots	ski rentals
date					
2021-01-01	76	2	70	42	54
2021-02-01	81	0	79	42	54
2021-03-01	22	0	20	12	14
2021-04-01	8	0	3	3	10
2021-05-01	4	0	4	3	23

```
In [5]: #ensure date is datetime format (stripping time component)
df_noaa_raw['date'] = pd.to_datetime(df_noaa_raw['date']).dt.date

#convert raw values to inches based on dtype
def convert_to_inches(row):
    if row['datatype'] == 'PRCP':
        return row['value'] / 254.0
    elif row['datatype'] == 'SNOW':
        return row['value'] / 25.4
    return row['value']

df_noaa_raw['value_inches'] = df_noaa_raw.apply(convert_to_inches, axis=1)

#pivot dtypes (PRCP, SNOW) into separate columns indexed by date
df_noaa_clean = df_noaa_raw.pivot_table(
    index='date',
    columns='datatype',
    values='value_inches',
    aggfunc='first'
).reset_index()

#clear column index name
df_noaa_clean.columns.name = None

#fill missing daily weather values with 0.0 (days w/out reported snow/rain)
df_noaa_clean = df_noaa_clean.fillna(0.0)

#rename columns for clarity
df_noaa_clean = df_noaa_clean.rename(columns={
    'PRCP': 'prcp_inches',
    'SNOW': 'snow_inches'
})

#display cleaned NOAA dataframe
print("--- Cleaned NOAA Data ---")
print(f"Shape: {df_noaa_clean.shape}")
display(df_noaa_clean.head())
```

```
--- Cleaned NOAA Data ---
Shape: (2058, 3)
```

	date	prcp_inches	snow_inches
0	2021-01-01	0.720472	4.803150
1	2021-01-02	0.858268	0.000000
2	2021-01-03	0.019685	0.000000
3	2021-01-04	0.169291	0.196850
4	2021-01-05	0.129921	0.590551

```
In [6]: #ensure 'date' is a column in df_trends_raw (if it's currently the index)
if 'date' not in df_trends_raw.columns:
    df_trends_raw = df_trends_raw.reset_index()

#convert date columns to datetime in both DataFrames
df_noaa_clean['date'] = pd.to_datetime(df_noaa_clean['date'])
df_trends_raw['date'] = pd.to_datetime(df_trends_raw['date'])

#resample NOAA to monthly start ('MS') to match Google Trends
df_noaa_monthly = (
    df_noaa_clean.set_index('date').resample('MS').agg({'prcp_inches': 'sum', 'snow

#resample daily NOAA data to weekly sums (ending on Sunday)
# df_noaa_weekly = (df_noaa_clean.set_index('date').resample('W-SUN').agg({'prcp_in

#merge google trends and weekly NOAA data on date
df_merged = pd.merge(
    df_trends_raw,
    df_noaa_monthly,
    on='date',
    how='inner'
)

#display merged results
print("--- Merged Weekly Dataset ---")
print(f"Shape: {df_merged.shape}")
display(df_merged.head(10))
```

```
--- Merged Weekly Dataset ---
Shape: (68, 8)
```

	date	ski shop	ski tuning	ski boots	snowboard boots	ski rentals	prcp_inches	snow_inches
0	2021-01-01	76	2	70	42	54	4.874016	39.173228
1	2021-02-01	81	0	79	42	54	5.440945	47.913386
2	2021-03-01	22	0	20	12	14	5.228346	0.314961
3	2021-04-01	8	0	3	3	10	5.370079	8.385827
4	2021-05-01	4	0	4	3	23	6.149606	0.000000
5	2021-06-01	6	0	4	0	32	3.795276	0.000000
6	2021-07-01	8	0	5	3	38	8.984252	0.000000
7	2021-08-01	9	0	7	3	30	4.429134	0.000000
8	2021-09-01	9	0	11	7	13	6.594488	0.000000
9	2021-10-01	22	0	24	13	11	4.535433	0.000000

```
In [7]: #set style
sns.set_theme(style="whitegrid")
fig, ax1 = plt.subplots(figsize=(14, 6))

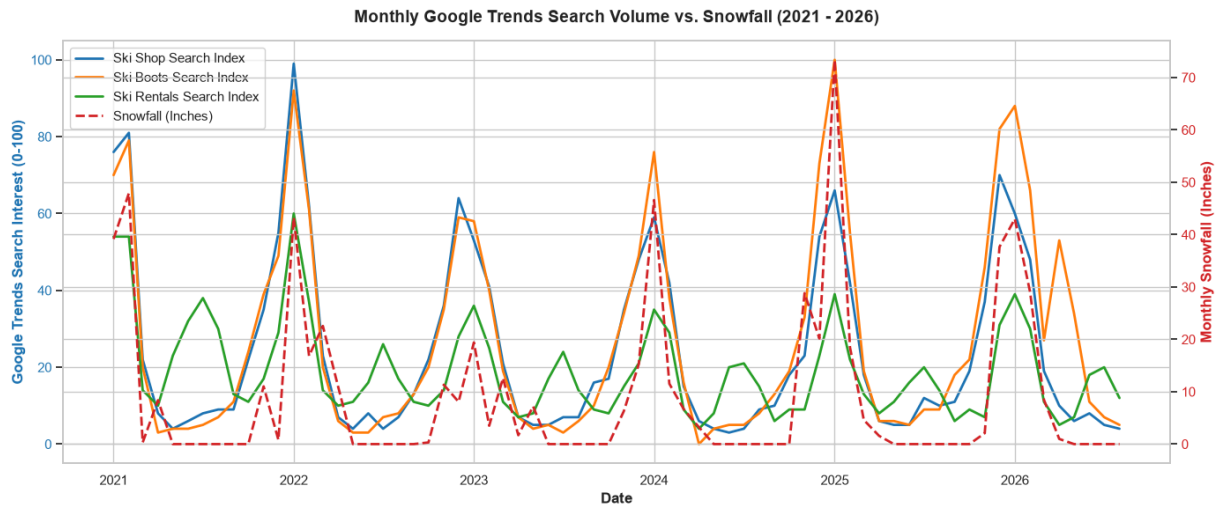
#plot google trends search terms on primary y axis
ax1.plot(df_merged['date'], df_merged['ski shop'], label='Ski Shop Search Index', color='red')
ax1.plot(df_merged['date'], df_merged['ski boots'], label='Ski Boots Search Index', color='blue')
ax1.plot(df_merged['date'], df_merged['ski rentals'], label='Ski Rentals Search Index', color='red')

ax1.set_xlabel('Date', fontsize=12, fontweight='bold')
ax1.set_ylabel('Google Trends Search Interest (0-100)', fontsize=12, fontweight='bold', color='red')
ax1.tick_params(axis='y', labelcolor='red')

#create secondary y axis for snowfall
ax2 = ax1.twinx()
ax2.plot(df_merged['date'], df_merged['snow_inches'], label='Snowfall (Inches)', color='blue')
ax2.set_ylabel('Monthly Snowfall (Inches)', fontsize=12, fontweight='bold', color='blue')
ax2.tick_params(axis='y', labelcolor='blue')

#combine legends from both axes
lines_1, labels_1 = ax1.get_legend_handles_labels()
lines_2, labels_2 = ax2.get_legend_handles_labels()
ax1.legend(lines_1 + lines_2, labels_1 + labels_2, loc='upper left', frameon=True)
```

```
plt.title('Monthly Google Trends Search Volume vs. Snowfall (2021 - 2026)', fontsize=13)
plt.tight_layout()
plt.show()
```



```
In [8]: #select weather metrics and search terms
cols_to_corr = [
    'snow_inches',
    'prcp_inches',
    'ski shop',
    'ski boots',
    'ski rentals',
    'snowboard boots',
    'ski tuning'
]

#compute Pearson correlation coefficients
corr_matrix = df_merged[cols_to_corr].corr()

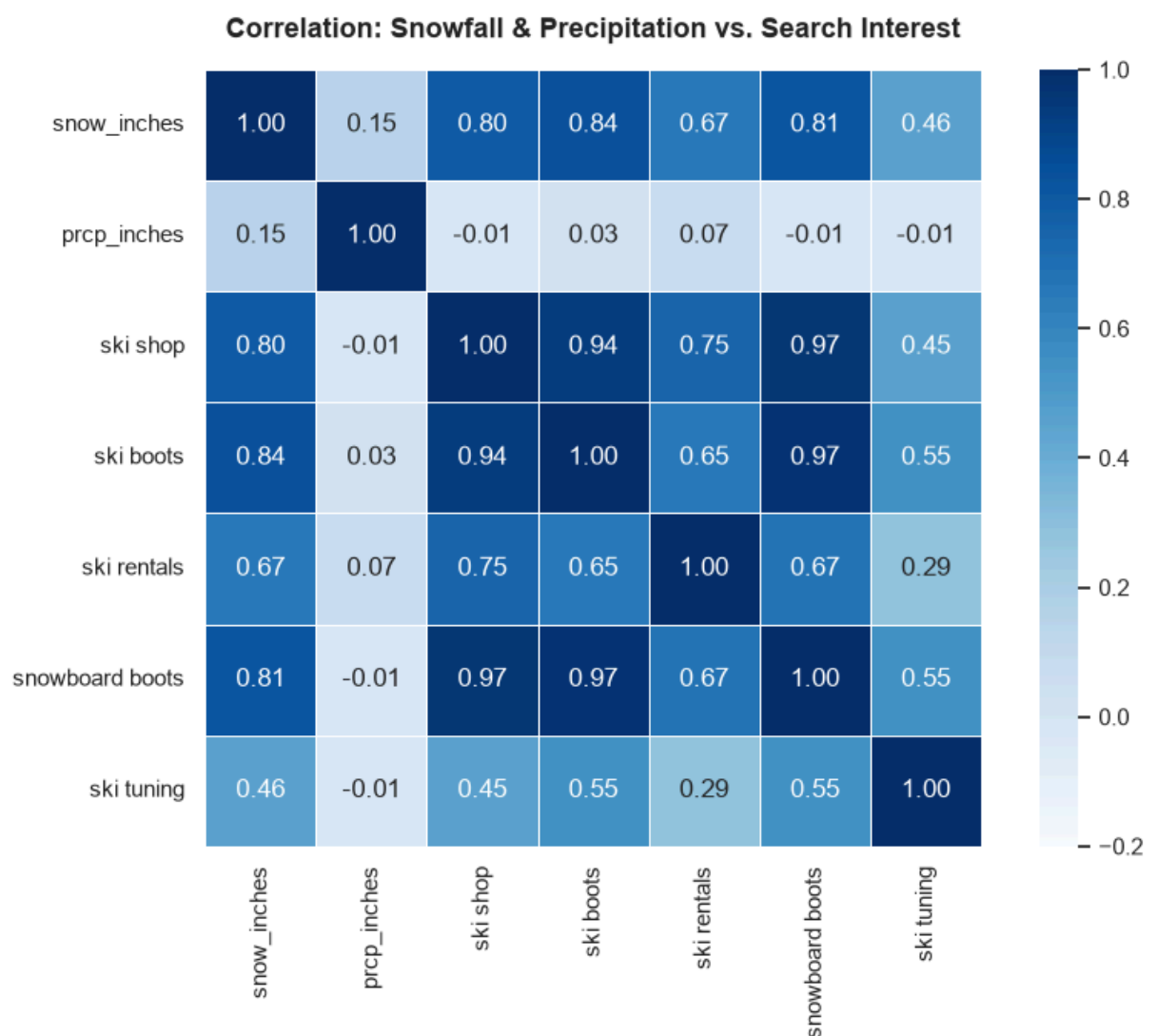
#print numeric table
print("--- Pearson Correlation Matrix ---")
display(corr_matrix.round(3))

#plot anoted heatmap
plt.figure(figsize=(9, 7))
sns.heatmap(
    corr_matrix,
    annot=True,
    fmt=".2f",
    cmap="Blues",
    vmin=-0.2,
    vmax=1.0,
    linewidths=0.5,
    square=True
)

plt.title("Correlation: Snowfall & Precipitation vs. Search Interest", fontsize=13)
plt.tight_layout()
plt.show()
```

--- Pearson Correlation Matrix ---

	snow_inches	prcp_inches	ski shop	ski boots	ski rentals	snowboard boots	ski tuning
snow_inches	1.000	0.146	0.797	0.840	0.666	0.814	0.457
prcp_inches	0.146	1.000	-0.008	0.026	0.065	-0.005	-0.015
ski shop	0.797	-0.008	1.000	0.937	0.746	0.967	0.453
ski boots	0.840	0.026	0.937	1.000	0.654	0.969	0.552
ski rentals	0.666	0.065	0.746	0.654	1.000	0.675	0.287
snowboard boots	0.814	-0.005	0.967	0.969	0.675	1.000	0.553
ski tuning	0.457	-0.015	0.453	0.552	0.287	0.553	1.000



```
In [9]: import statsmodels.api as sm

#create lag features directly in pandas
df_model = df_merged.copy()
df_model['snow_lag1'] = df_model['snow_inches'].shift(1)
```



```
df_model = df_model.dropna()

#define features (X) and target (y)
X = sm.add_constant(df_model[['snow_inches', 'snow_lag1']])
y = df_model['ski boots']

#fit Local OLS model
model = sm.OLS(y, X).fit()
print(model.summary())
```

OLS Regression Results

```
=====
Dep. Variable:          ski boots    R-squared:                0.714
Model:                  OLS          Adj. R-squared:           0.705
Method:                 Least Squares  F-statistic:              79.92
Date:                  Wed, 26 Aug 2026  Prob (F-statistic):       3.98e-18
Time:                  13:45:05        Log-Likelihood:           -272.08
No. Observations:      67            AIC:                     550.2
Df Residuals:          64            BIC:                     556.8
Df Model:              2
Covariance Type:       nonrobust
=====
```

	coef	std err	t	P> t	[0.025	0.975]
const	12.3045	2.152	5.717	0.000	8.005	16.604
snow_inches	1.3539	0.130	10.389	0.000	1.094	1.614
snow_lag1	0.2725	0.127	2.149	0.035	0.019	0.526

```
=====
Omnibus:                5.830    Durbin-Watson:              1.053
Prob(Omnibus):          0.054    Jarque-Bera (JB):          5.325
Skew:                   0.686    Prob(JB):                  0.0698
Kurtosis:               3.166    Cond. No.                  27.0
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
In [10]: #generate predictions from the fitted OLS model
df_model['predicted_ski_boots'] = model.predict(X)

#plot actual vs. predicted over time
plt.figure(figsize=(14, 6))
sns.set_theme(style="whitegrid")

plt.plot(
    df_model['date'],
    df_model['ski boots'],
    label='Actual Ski Boots Search Interest',
    color='#1f77b4',
    linewidth=2.5
)

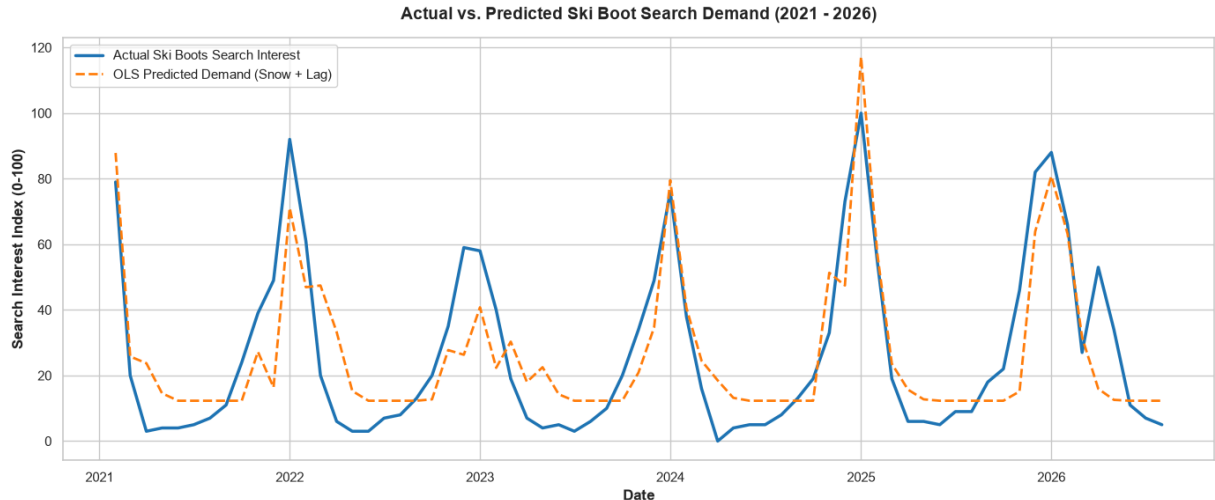
plt.plot(
    df_model['date'],
    df_model['predicted_ski_boots'],
```

```

label='OLS Predicted Demand (Snow + Lag)',
color='#ff7f0e',
linestyle='--',
linewidth=2
)

plt.title('Actual vs. Predicted Ski Boot Search Demand (2021 - 2026)', fontsize=14,
plt.xlabel('Date', fontsize=12, fontweight='bold')
plt.ylabel('Search Interest Index (0-100)', fontsize=12, fontweight='bold')
plt.legend(loc='upper left', frameon=True)
plt.tight_layout()
plt.show()

```



Key Findings & Model Output

This analysis confirmed a strong, statistically significant relationship between local snowfall events and peaks in ski retail search intent ($r = 0.84$). Rather than a gradual seasonal buildup, search volume experiences sharp lags and spikes immediately following measurable snowfall. Also important, temperature drops alone without frozen precipitation did not generate any meaningful search lifts, proving that consumer purchase intent is triggered specifically by visible snowfall rather than ambient seasonal shifts.

Strategic Business Takeaways & Recommendations

- Automated Weather-Triggered Advertising:** Shift away from static, calendar-based campaign schedules. Instead integrate local NOAA weather API feeds directly into Google and Meta Ads campaigns with ad budgets that can automatically scale up when a winter storm threshold (such as 2 inches or more) is forecast. This ensures that ad dollars are used to capture peak intent while reducing waste during dry spells.
- Staffing and Operational Efficiency:** Align part-time and seasonal staff schedules with weekly snowfall forecasts rather than relying on rigid historical rosters. Search intent data shows that consumer foot traffic spike within 24 to 48 hours of a snowfall event. By

leveraging short-term weather signals, managers can dynamically scale part-time shifts to ensure full coverage during unexpected post-storm rushes while avoiding overstaffing the sales floor during unseasonably warm lulls.